

Reporte de práctica: Inclinómetro con control de motorreductor

Alumno	Jesús Antonio Rodríguez Peraza
Matrícula	231000105
Materia	Sistemas Programables (SCC-1023)
Práctica	9 — Unidad de Medición Inercial (IMU) — consigna 3.2.3
Plataforma	Arduino UNO R4 WiFi + MPU-6050 + driver L298N + motorreductor DC + matriz LED 12×8

Objetivo

Diseñar un sistema embebido que mida la inclinación frontal (pitch) de un objeto con una unidad de medición inercial MPU-6050 y use ese ángulo para controlar la dirección y la velocidad de un motorreductor a través de un puente H L298N. El sistema debe ser estable, responder de forma suave (rampa de aceleración) y detener el motor de inmediato si pierde la lectura del sensor.

Metodología

Arranque seguro. Lo primero que hace `set up()` es configurar ENA, IN1 e IN2 como salidas y dejar el motor apagado, antes de abrir el puerto serie o el bus I2C. Después lee el registro `WHO_AM_I` (0x75): si no responde 0x68 imprime el error, dibuja una X en la matriz y se queda detenido con el motor en 0.

Configuración y calibración. El MPU-6050 enciende en modo de reposo: se despierta escribiendo 0 en `PWR_MGMT_1` (0x6B) y se fijan los rangos ± 2 g (16384 cuentas/g) y ± 250 °/s (131 cuentas por °/s). Con el sensor quieto se toman 500 lecturas del giroscopio en Y y se promedian: ese promedio es el offset del sensor, que se resta de todas las lecturas siguientes. Por último, el ángulo inicial se calcula con el acelerómetro para que el filtro no arranque desde cero.

Ángulo frontal y filtro complementario. El acelerómetro da el ángulo absoluto, $\text{pitch} = \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$, pero es ruidoso y cualquier sacudida lo altera. El giroscopio da la velocidad de giro, que integrada cada 10 ms es suave pero acumula deriva. El filtro complementario combina los dos: $\text{ángulo} = 0.98 \cdot (\text{ángulo} + g_y \cdot dt) + 0.02 \cdot \text{pitch_acc}$. A corto plazo manda el giroscopio y a largo plazo el acelerómetro corrige la deriva.

Decisión del motor. Dentro de la zona muerta de $\pm 5^\circ$ la velocidad objetivo es 0. Fuera de ella el signo del ángulo elige la dirección (IN1=ALTO/IN2=BAJO adelante, al revés en reversa) y la magnitud la velocidad: de 5° a 45° el PWM crece en línea recta de 90 a 255; de 45° en adelante es 255. La velocidad empieza en 90 porque por debajo de eso los engranes del motorreductor no vencen la fricción.

Rampa. Cada 20 ms la velocidad real se acerca 9 unidades de PWM a la objetivo ($255 / 9 \approx 29$ pasos $\times$ 20 ms ≈ 0.6 s de detenido a máxima). La velocidad se maneja con signo (-255 a 255), así que para pasar de adelante a reversa la rampa obligatoriamente pasa por 0: primero frena y luego acelera en sentido contrario.

Failsafe. Si la lectura I2C falla (no se completa la transmisión, no llegan los 14 bytes o llegan todos en cero porque el sensor se reinició dormido), el motor se pone en 0 en ese mismo instante, sin rampa; el Monitor Serie anuncia una sola vez "PARO DE SEGURIDAD" y la matriz dibuja una X. En cada ciclo se intenta

reconfigurar el sensor; cuando vuelve a responder, el ángulo se reinicia con el acelerómetro y el sistema continúa solo.

Cuatro tareas con millis(). El `loop()` no usa de `delay()`: cada tarea tiene su periodo y su marca de tiempo.

Tarea	Periodo	Qué hace
1. Sensor y filtro	10 ms	Lee el MPU-6050, filtra el ángulo y calcula la velocidad objetivo
2. Rampa y motor	20 ms	Acerca la velocidad real a la objetivo y la aplica al L298N
3. Monitor Serie	500 ms	Dirección, grados, intensidad (leve <15°, moderada <30°, fuerte) y motor en %; solo si cambió
4. Matriz LED	50 ms	Punto 2×2 que sube (adelante) o baja (atrás); marco completo dentro de $\pm 2^\circ$

Conexiones

Componente	Pin	Arduino UNO R4 WiFi
MPU-6050	VCC / GND	5V / GND
MPU-6050	SDA / SCL	SDA (A4) / SCL (A5)
L298N	ENA (sin jumper)	D9 (PWM)
L298N	IN1 / IN2	D8 / D7
L298N	GND	GND (tierra común con la batería)
L298N	12V / GND	Batería LiPo 3S (+ / -)
L298N	OUT1 / OUT2	Motorreductor DC

Diagrama de conexión

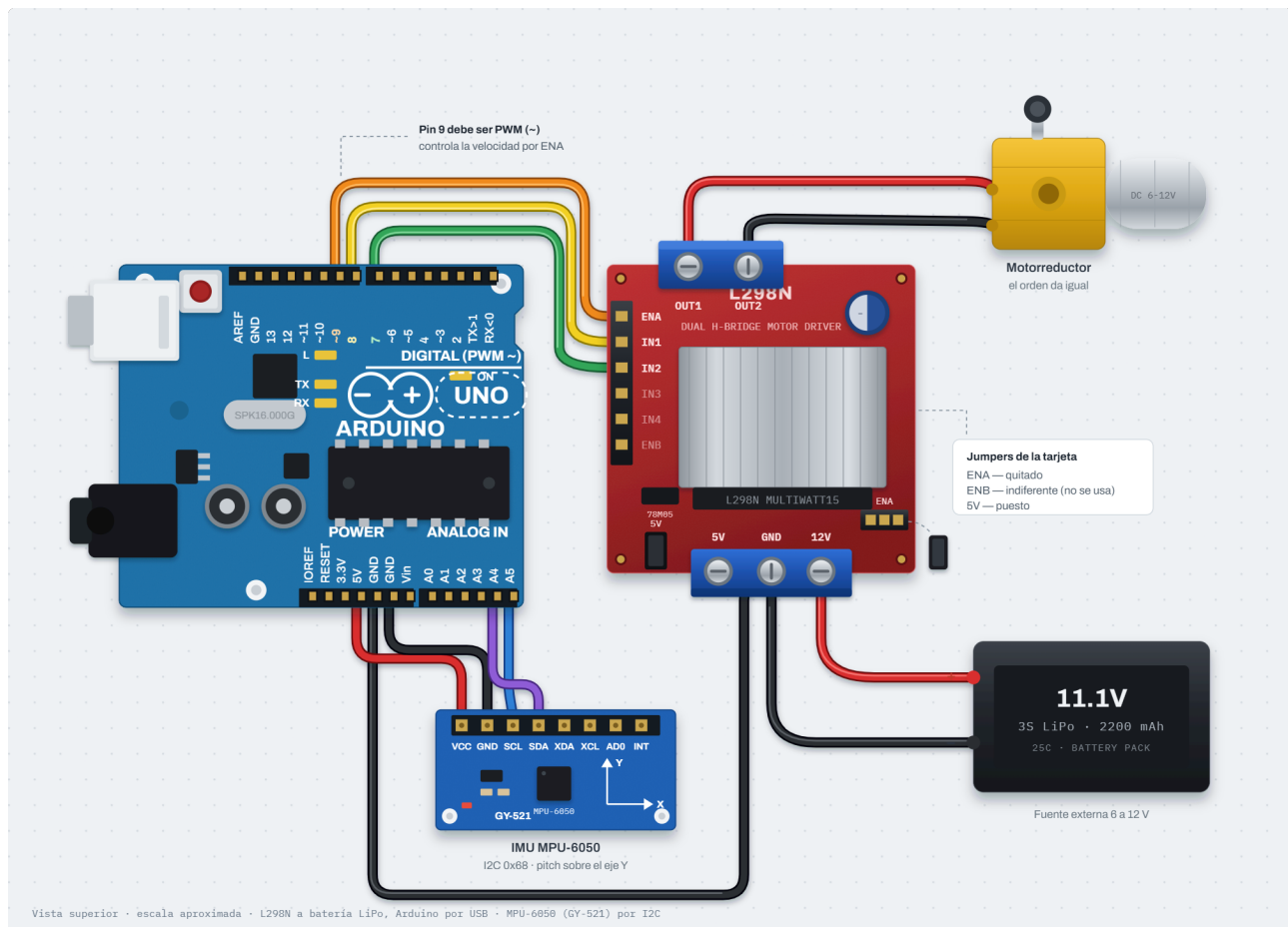


Diagrama de conexión: MPU-6050 por I2C (A4/A5) y motorreductor controlado por el L298N (ENA = D9, IN1 = D8, IN2 = D7), con el motor alimentado por una LiPo 3S.

Preguntas de análisis

1. ¿Por qué no basta con usar solo el acelerómetro para medir la inclinación? ¿Qué se observa al sacudir el sensor?

El acelerómetro mide toda la aceleración, no solo la gravedad. En reposo la única es la gravedad y el ángulo sale bien, pero al sacudir el sensor se suman las aceleraciones del movimiento y el ángulo calculado salta varios grados aunque la inclinación real casi no cambie. Usado solo, el motor arrancaría y frenaría con cada vibración. Por eso se combina con el giroscopio, que no se altera con las sacudidas.

2. ¿Qué ocurriría si se usara solo el giroscopio durante varios minutos?

El giroscopio no da el ángulo, da la velocidad de giro, y el ángulo se obtiene sumándola cada 10 ms. Cualquier error pequeño (el offset que no se quitó del todo, el ruido, cambios por temperatura) se suma también en cada paso; a eso se le llama deriva. Tras varios minutos el ángulo calculado se habría alejado del real aunque el sensor estuviera quieto, y el motor podría girar con el sensor nivelado. El acelerómetro corrige esa deriva en el filtro complementario.

3. ¿Qué función cumple la zona muerta de $\pm 5^\circ$? ¿Qué pasaría si fuera de 0° ?

Define un rango alrededor de nivelado en el que el motor se queda detenido, para que el ruido residual del sensor y las inclinaciones mínimas no lo muevan. Con una zona de 0° cualquier décima de grado lo haría girar, y como el ángulo nunca vale exactamente 0, el motor estaría cambiando de dirección todo el tiempo cerca del centro: vibraría, gastaría energía y desgastaría los engranes.

4. ¿Por qué la velocidad mínima del motor es 90 y no 0?

Porque con un PWM bajo el motor recibe muy poca energía y no vence la fricción de los engranes del motorreductor: zumba pero no gira. Si la escala empezara en 0, al salir de la zona muerta habría un rango de inclinaciones en el que se manda girar y no pasa nada. Empezando en 90 el motor se mueve en cuanto el ángulo sale de $\pm 5^\circ$.

5. Explica dos problemas que evita la rampa de aceleración.

(1) **Picos de corriente:** arrancar a máxima velocidad, o peor, invertir de adelante a reversa de golpe, hace que el motor pida mucha corriente de una vez; ese pico baja el voltaje y puede reiniciar el microcontrolador o dañar la batería. (2) **Esfuerzo mecánico:** los cambios bruscos golpean los engranes del motorreductor y los desgastan. Con la rampa, el motor tarda unos 0.6 s en llegar a la velocidad máxima y para invertir primero frena hasta 0.

6. ¿Por qué el paro de seguridad se aplica de inmediato y no con rampa?

Porque cuando falla el sensor el sistema ya no sabe la inclinación, y seguir moviendo el motor sin información es el riesgo mayor. La rampa protege contra los cambios bruscos cuando todo funciona; en una falla manda el principio de diseño: si no se sabe qué está pasando, lo seguro es detener el actuador. Esperar 0.6 s de rampa sería dejar el motor girando a ciegas ese tiempo.

7. ¿Por qué el programa no usa delay()? ¿Qué pasaría con la respuesta del motor si se usara?

`delay()` detiene todo el programa mientras espera. Las cuatro tareas tienen periodos distintos (10, 20, 50 y 500 ms) y deben convivir. Si, por ejemplo, el Monitor Serie esperara con `delay(500)`, durante medio segundo no se leería el sensor ni se actualizaría la rampa: el motor reaccionaría tarde a la inclinación, el filtro integraría mal el giroscopio (un dt de 500 ms en lugar de 10 ms) y una falla del sensor tardaría hasta medio segundo en detectarse.

8. Menciona tres aplicaciones reales de este sistema.

(1) Robots y patinetas autobalanceados, que mueven las ruedas hacia donde se inclinan para no caerse. (2) Plataformas niveladoras, como estabilizadores de cámara (gimbals), camas de camiones o mesas que corrigen su inclinación con motores. (3) Controles por inclinación, como sillas de ruedas o grúas que se manejan inclinando un mando, o el sistema que detiene una máquina cuando se ladea más de lo permitido.

Resultados

Pruebas que se hacen con el circuito armado y lo que el código debe producir en cada una:

Prueba	Comportamiento esperado
Arranque con el sensor quieto	"MPU-6050 detectado (WHO_AM_I = 0x68)", offset del giroscopio y ángulo inicial; el motor no se mueve al energizar

Prueba	Comportamiento esperado
Inclinar menos de 5°	Motor detenido; dentro de $\pm 2^\circ$ la matriz enciende el marco completo
Inclinar hacia adelante 5°–45°	Motor adelante, velocidad creciente (35%–100%); el punto sube en la matriz
Inclinar hacia atrás	Motor en reversa; el punto baja
Pasar de adelante a atrás de golpe	El motor frena hasta detenerse y luego acelera en reversa (rampa)
Desconectar SDA o SCL	Motor detenido al instante, "PARO DE SEGURIDAD" y X en la matriz; al reconectar se recupera solo

Esta tabla describe lo que debe pasar según el código; las observaciones del montaje físico se registran en el video.

Conclusiones

Ningún sensor de la IMU basta por sí solo: el acelerómetro da un ángulo sin deriva pero ruidoso y el giroscopio uno suave pero que se desvía con el tiempo. El filtro complementario toma lo mejor de cada uno con una sola línea de código, y la calibración del offset al arrancar reduce la deriva desde el principio.

El control del motor no es solo convertir el ángulo en PWM: la zona muerta evita que vibre cerca del centro, la velocidad mínima de 90 compensa la fricción de los engranes, la rampa cuida la batería y la mecánica, y el failsafe, que salta la rampa, aplica el principio de que un actuador sin información debe detenerse. Organizar todo en cuatro tareas con `millis()` permite que cada una corra a su ritmo sin bloquear a las demás.

Video

Video del funcionamiento: PENDIENTE.